

DIVERSITY ENHANCES AN LLM’S PERFORMANCE IN RAG AND LONG-CONTEXT TASK

Zhichao Wang*, Bin Bi, Yanqi Luo, Sitaram Asur, Claire Na Cheng
Salesforce
zhichaowang@salesforce.com

ABSTRACT

The rapid advancements in large language models (LLMs) have highlighted the challenge of context window limitations, primarily due to the quadratic time complexity of the self-attention mechanism ($O(N^2)$, where N denotes the context window length). This constraint impacts tasks such as retrieval-augmented generation (RAG) in question answering (Q&A) and long context summarization. A common approach involves selecting content with the highest similarity to the query; however, this often leads to redundancy and the exclusion of diverse yet relevant information. Building on principles from Maximal Marginal Relevance (MMR) and Farthest Point Sampling (FPS), we integrate diversity into the content selection process. Our findings reveal that incorporating diversity substantially increases the recall of selecting relevant sentences or chunks before LLM-based Q&A and summarization. These results highlight the importance of maintaining diversity in future LLM applications to further improve summarization and Q&A outcomes.

1 Introduction

The remarkable success of Transformer models [1], BERT [2], and GPT [3] can be largely attributed to their robust self-attention mechanisms. However, the self-attention module’s quadratic time complexity, $O(N^2)$, where N represents the context window length, has imposed limitations on the size of the context window.

Recent advances in LLMs have partially addressed this constraint. For instance, GPT-3.5 demonstrates the capability to process context windows of up to 16,385 tokens, while GPT-4 extends this capacity to an impressive 128,000 tokens. Despite these notable improvements, the challenge of processing even longer sequences remains a critical area of research for several compelling reasons. First, many real-world applications, such as question-answering systems operating on extensive datasets, cannot accommodate entire document collections within the LLM’s context window. This limitation has led to the development of Retrieval-Augmented Generation (RAG) systems [4], which selectively retrieve and process relevant text segments for specific queries. Second, while current context window sizes may suffice for conventional Natural Language Processing (NLP) tasks, they prove inadequate for high-frequency signal processing applications. For example, audio processing and medical vibrational signal analysis often require handling data streams with sampling rates reaching one million samples per second, far exceeding current context window capabilities [5]. Furthermore, empirical studies have revealed a concerning trend: LLM performance tends to degrade as input lengths approach the maximum context window capacity, highlighting the need for more robust solutions to long-sequence processing [6].

Various strategies have been devised to address the limited context window issue in LLMs. Longformer [7] applies attention to immediate local neighbors, reducing the time complexity from $O(N^2)$ to $O(NM)$, with M representing the considered neighbors. An alternative strategy is to expand the context window at inference [8]. Lastly, a more straightforward approach is to first select the most relevant documents while ensuring they fit within the LLM’s context window. For a given query, multiple documents are split into smaller chunks or sentences. The embeddings for both the query and the split documents are then computed. Similarity metrics, such as cosine similarity or Euclidean distance, are subsequently used to identify the most relevant sentences.

*: corresponding author

github code: <https://github.com/ZhichaoWang970201/DIVERSITY-ENHANCES-LLM>

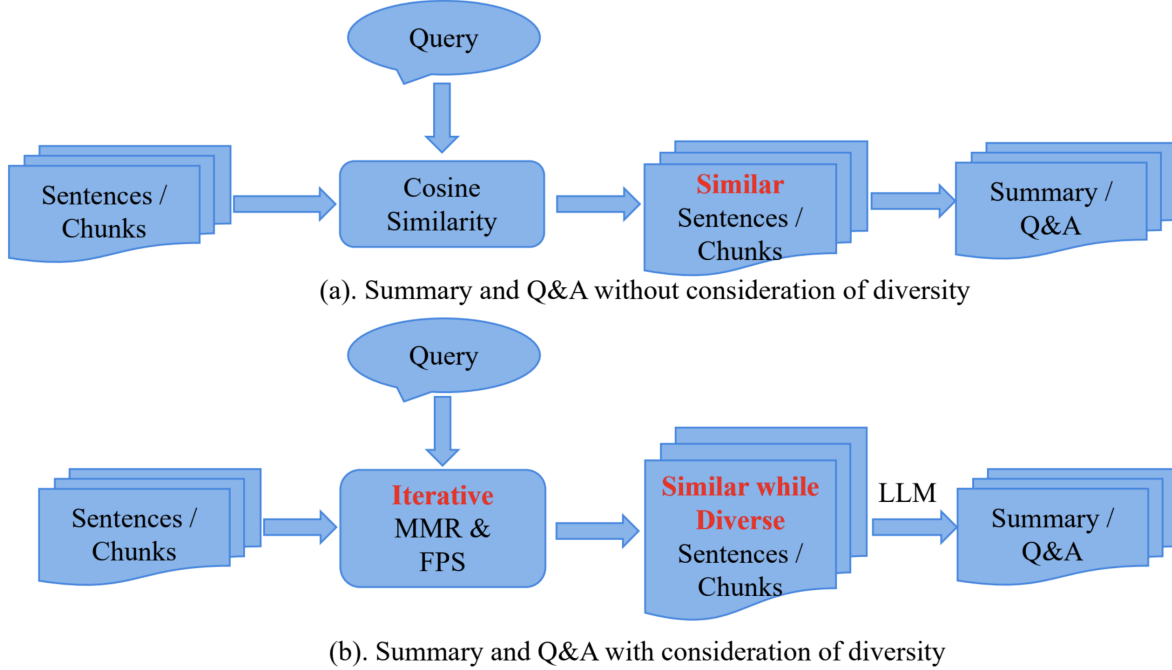


Figure 1: For both Q&A and summarization tasks, the initial dataset is divided into sentences or chunks, and corresponding embeddings are extracted. In a traditional pipeline, query embeddings are generated and used to select relevant materials to LLMs for downstream tasks. In contrast, methods like MMR and FPS incorporate diversity in a greedy manner when selecting relevant sentences. This approach increases the recall of the correct answer within the chosen sentences or chunks.

However, relying solely on the similarity between a query and segmented documents can result in overlooking critical information due to excessive focus on similar content. Previous studies have introduced greedy algorithms, such as MMR [9] and FPS [10], to improve diversity during the selection process. In this paper, we aim to demonstrate the significance of diversity in long context summarization and RAG-based Q&A tasks at multiple levels: sentence-level for single documents, chunk-level across entire datasets, and sentence-level in summarization.

The contributions of this paper are summarized as follows: *WOSTO*

1. We demonstrate the benefits of diversity using MMR and FPS with proper hyperparameters, i.e., α and w on downstream tasks, including Q&A and summarization.
2. We discover that MMR achieves slightly better recall than FPS while maintaining significantly lower latency.
3. We prove the ordering selected sentences within the original document and ordering selected chunks based on the scores has the best downstream performances.

2 Methodology

In this section, we will start with a brief introduction of MMR and FPS to consider diversity during the search process. Then, the integration with LLM will be discussed.

2.1 FPS and MMR for Diversity

FPS The concept of Farthest Point Sampling (FPS) originates from the field of 3D computer vision [10]. Its primary goal is to select a diverse set of points from a given point cloud, which facilitates hierarchical feature extraction for downstream applications. The FPS process begins with a randomly selected initial point. In each subsequent iteration, a new point is chosen based on its maximum distance from all previously selected points as shown in Eq. 1 where S refers to the selected set, R refers to the remaining set and $dist(i, j)$ represents the Euclidean distance between i and j . For initialization, the selected set S is empty and the remaining set R equals to the entire dataset. An illustration of FPS

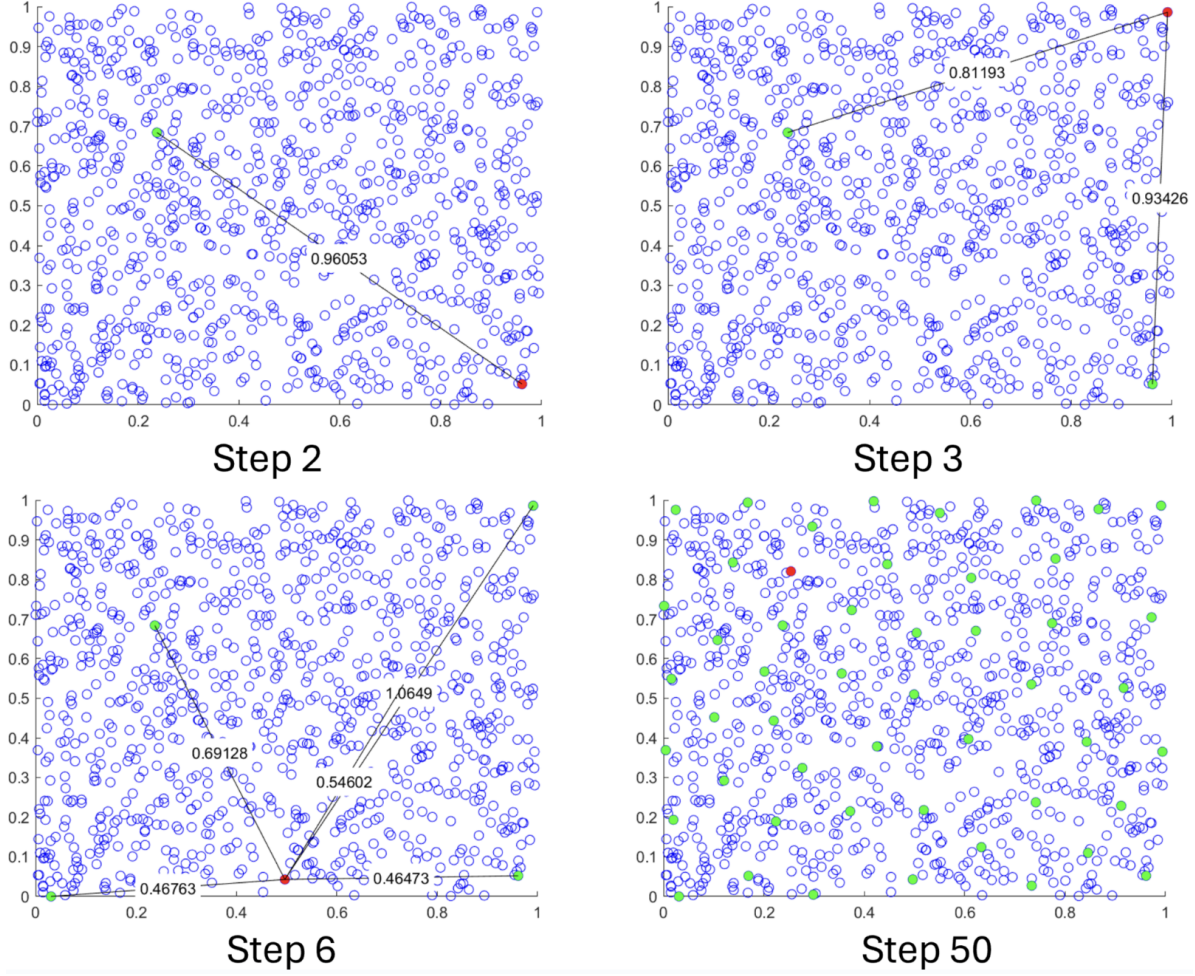


Figure 2: An example of FPS: selecting 50 points from 1000 points in 2D space. red: currently selected point; green: previous selected points; blue: unselected points

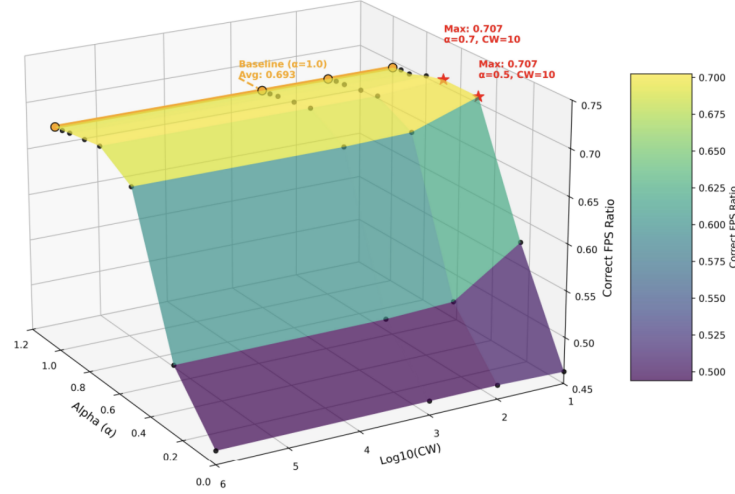
is provided in Figure 2, where 50 points are selected from a set of 1000 points in a 2D domain. Unlike some other selection methods, FPS solely relies on Euclidean distance to promote diversity among the selected points.

$$\operatorname{argmax}_{i \in R} \left[\max_{j \in S} \operatorname{dist}(i, j) \right] \quad \text{max with dist} \quad (1)$$

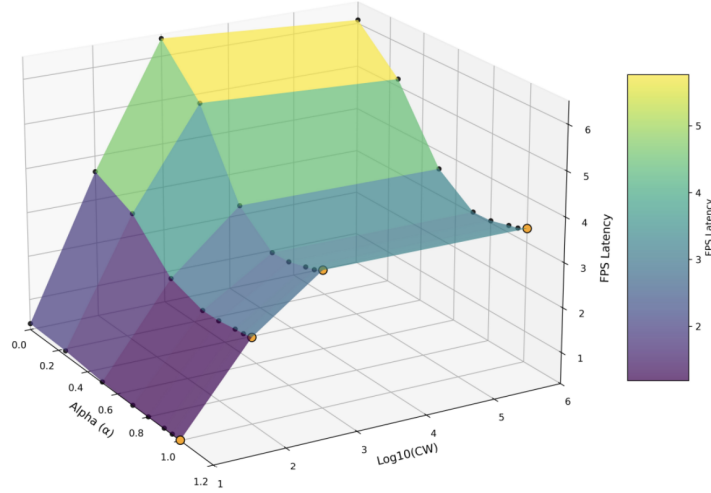
MMR In contrast, Maximal Marginal Relevance (MMR) selects a subset S from a large dataset with the dual objectives of maximizing rewards and ensuring diversity [9]. At each iteration, an element is chosen based on a locally optimal criterion defined in Eq. 2. Here, the reward r_i of the i -th item is balanced against the similarity measure, where $\cos(i, j)$ denotes the cosine similarity between the i -th item and those in a context window W (comprising the most recently selected examples). This context window, for example, could include the last 10 selected samples when $w = 10$, reducing the influence of earlier selections. The iterative process continues until a predefined termination criterion, such as reaching a maximum number of tokens, is met.

$$\operatorname{argmax}_{i \in R} \left[\alpha \cdot r_i - (1 - \alpha) \cdot \max_{j \in W} \cos(i, j) \right] \quad (2)$$

When comparing the two methods, both FPS and MMR use greedy selection to promote diversity. The key differences lie in their selection criteria: while FPS focuses exclusively on maximizing Euclidean distance for diversity, MMR introduces an additional reward factor and employs cosine similarity within a dynamic context window to balance



(a). FPS accuracy for different hyperparameters



(b). FPS latency for different hyperparameters

Figure 3: The impact of different hyperparameters: α , w , c_r on the recall accuracy and latency using FPS of the Natural Question dataset of single document Q&A.

between relevance and diversity. In this paper, we extend FPS by incorporating the reward mechanism and the dynamic context window. Ultimately, the key distinction between the modified FPS and MMR lies in their similarity metrics: the former uses Euclidean distance, while the latter relies on cosine similarity.

2.2 Combine MMR and Modified FPS with LLM for Diversity on Q&A and Summarization

Extending MMR and Modified FPS techniques for LLMs in tasks such as Q&A and summarization is relatively straightforward as shown in Fig. 1. These techniques employ a greedy approach to iteratively balance the similarity of selected sentences or chunks to the query with the diversity among the selected sentences or chunks. This method enhances the likelihood of selecting the most relevant sentences or chunks for LLMs in downstream tasks. Lastly, inspired by [11], a heuristic rearrangement scheme is implemented to enhance the likelihood of identifying the correct answer from the retrieved documents.

Q&A To evaluate the ability of LLMs on accurately extracting the correct answer, a query, a document, and a corresponding answer are initially provided. Documents are pre-processed by dividing them into sentences or chunks,

Q&A	Natural Question			Trival Q&A			Narrative Q&A		
	$c_r=0.05$	$c_r=0.1$	$c_r=0.2$	$c_r=0.05$	$c_r=0.1$	$c_r=0.2$	$c_r=0.05$	$c_r=0.1$	$c_r=0.2$
SB	46.28	58.60	69.41	63.44	71.64	78.33	18.60	21.04	25.61
SB+MMR	50.43	63.18	72.47	65.29	74.02	80.47	20.88	24.09	27.59
SB+FPS	50.88	63.23	72.33	65.25	73.18	80.07	21.34	24.24	27.44

Table 1: This table compares the performance of SB+MMR and SB+FPS against SB across three different datasets and three compression ratios, focusing on the recall of the correct answer within the selected documents.

Q&A	Natural Question			Trival Q&A			Narrative Q&A		
	$c_r=0.05$	$c_r=0.1$	$c_r=0.2$	$c_r=0.05$	$c_r=0.1$	$c_r=0.2$	$c_r=0.05$	$c_r=0.1$	$c_r=0.2$
SB (index sort)	40.44	51.74	64.25	75.66	75.95	76.25	15.40	17.98	18.60
SB (sort)	40.25	50.81	60.61	75.76	76.15	76.38	13.72	15.55	17.07
SB (1:1)	40.25	51.55	60.24	75.29	76.33	76.47	14.18	16.31	17.99
SB (2:1)	41.28	50.99	61.08	75.31	76.33	76.87	14.18	16.46	17.84
SB (3:1)	39.41	51.09	60.05	75.56	76.13	76.57	14.63	16.01	17.23
SB+MMR (index sort)	45.76	57.25	67.90	75.73	76.20	76.72	18.60	19.05	20.27
SB+MMR (sort)	44.45	55.19	64.35	76.20	76.85	77.00	16.31	16.46	17.38
SB+MMR (1:1)	45.48	55.57	63.60	76.23	76.77	77.34	16.46	16.62	16.92
SB+MMR (2:1)	45.20	55.29	63.32	75.90	76.20	76.65	15.09	15.55	16.01
SB+MMR (3:1)	43.80	55.85	63.51	76.05	76.72	76.92	16.16	16.92	16.31
SB+FPS (index sort)	46.79	59.12	67.71	75.93	76.13	76.60	17.68	17.68	19.05
SB+FPS (sort)	45.76	57.25	63.32	75.83	76.45	76.87	16.31	16.62	17.23
SB+FPS (1:1)	46.14	57.90	62.76	75.78	76.35	77.09	16.62	17.07	16.77
SB+FPS (2:1)	45.76	56.13	62.20	75.88	76.23	77.02	15.85	16.62	16.46
SB+FPS (3:1)	44.27	55.10	63.69	76.40	76.77	76.95	15.70	16.46	16.01

Table 2: This table compares the performance of SB+MMR and SB+FPS against SB across three different datasets and three compression ratios, focusing on the recall of the correct answer within the LLM responses.

and their embeddings are extracted beforehand. Both the query and the segmented documents are processed using encoder-only models to generate embeddings. In MMR, similarity is measured using the cosine angle, whereas in Modified FPS, Euclidean distance is used to assess similarity. For benchmarking Q&A performance, two metrics should be evaluated:

1. Pre-LLM recall: whether the answer exists in the selected content before being sent to the LLM.
2. Post-LLM recall: whether the answer appears in the LLM’s output.

If the first metric shows significant improvement, the benefit of diversity becomes evident. Otherwise, the advantage of diversity may be limited. If the first metric improves while the second metric does not, it indicates that the performance of downstream tasks may be constrained by the capabilities of the LLM [11].

Summarization In summarization tasks, datasets typically consist of a document paired with a corresponding golden summary created by experts. When no specific query is provided, the process begins by dividing the document into manageable chunks. Encoder-only models are employed to generate embeddings for these chunks, and the mean of these embeddings is used to represent the query embedding. Following this, the same methodology as in the previous Q&A task is applied to extract content that optimizes both reward and diversity.

The selected chunks are ordered to align with their original sequence in the document. These ordered chunks are sent to the LLM for summarization. We recognize that evaluating the extracted content before it is submitted to the LLM for summarization may not be particularly meaningful. Instead, we assess the quality of the LLM-generated summary by comparing it to the golden summary using metrics such as ROUGE [12] or LLM-as-a-judge [13].

3 Experiments

The experiments conducted in this paper focus on three main topics: 1. Single Document Question Answering (Q&A), 2. Multiple Documents Question Answering (Q&A), and 3. Single Document Summarization.

For Single Document Q&A, the goal is to choose the correct answer from a set of candidate sentences within a single document. In Multiple Document Q&A, all documents in the dataset are firstly divided into chunks and then combined, and a query is used to find the correct answers across the entire dataset. Because the dataset size is too large,

approximation methods are used to enhance efficiency and speed. Specifically, two metrics are evaluated: 1. recall of the correct answer in the extracted document, and 2. recall of the correct answer in the LLM response. The benefit of diversity is primarily reflected in the improvement of the first metric, while performance improvements in Q&A and summarization are mainly indicated by the second metric.

For summarization, various hyperparameters are considered in the optimization process:

1. The weight balance between reward and diversity, denoted as α ,
2. The context window size, w ,
3. The compression ratio, c_r , or the maximum number of selected tokens, $T_{\max}$.

3.1 Single Document Q&A

For single document Q&A, three datasets are included: 1. Natural Question [14], 2. Trivial QA [15] and 3. Narrative QA [16]. For each dataset, it is composed of thousands of (query, document, answer) pairs where the answer exists within the document and answers the query. For each document, we split it into sentence using Spacy package [17]. SentenceBERT (SB) is utilized as the encoder to extract embeddings from sentences in different experiments [18]. Then, different compression ratios, i.e., $c_r = 0.05, 0.1, 0.2$ are utilized. Here, $c_r = 0.05$ represents that the fraction of the number of selected tokens over the total number of tokens should be 0.05, i.e., the termination condition when 0.05 of all tokens are selected for answering the query. As for α and w , different hyperparameters are tested in a two-level iteration. The first coarse-level iteration utilizes α from [0, 0.25, 0.5, 0.7, 0.8, 0.9, 0.95, 1] and w from [0, 10, 100, 1000, 1000000]. Then, the best performing from the first coarse-level iteration is selected. For the second granular-level iteration, it further divides the neighbors of the best performing first coarse-level hyperparameters and select the ones that have the best performance. An example of the best performing hyperparameters for FPS on recall accuracy and latency in the Natural Question dataset in the coarse level can be found in subfigure (a) of Fig. 3. More details on the impact of hyperparameters can be found in Fig. 6. In particular, for Natural question, $\alpha = 0.55$ and $w = 1$ is the best for MMR and $\alpha = 0.5$ and $w = 1$ is the best for FPS. For Narrative Q&A, $\alpha = 0.55$ and $w = 5$ is the best for both MMR and FPS. For Trivial Q&A, $\alpha = 0.6$ and $w = 3$ is the best for MMR and $\alpha = 0.7$ and $w = 1$ is the best for FPS.

Natural Question	GPT4			GPT3.5		
	$T_{\max}=120k$	$T_{\max}=50k$	$T_{\max}=20k$	$T_{\max}=10k$	$T_{\max}=5k$	$T_{\max}=2k$
E5	70.7	69.4	68.5	66.2	64.2	57.4
E5+MMR	71.5	71.5	69.8	67.2	65.1	57.8
Narrative Q&A	GPT4			GPT3.5		
	$T_{\max}=120k$	$T_{\max}=50k$	$T_{\max}=20k$	$T_{\max}=10k$	$T_{\max}=5k$	$T_{\max}=2k$
E5	13.42	10.06	6.7	4.88	4.88	4.57
E5+MMR	22.56	20.43	15.85	14.94	12.20	7.01
Trivial Q&A	GPT4			GPT3.5		
	$T_{\max}=120k$	$T_{\max}=50k$	$T_{\max}=20k$	$T_{\max}=10k$	$T_{\max}=5k$	$T_{\max}=2k$
E5	84.62	81.15	74.01	70.24	65.08	56.55
E5+MMR	88.99	85.81	82.24	78.57	74.01	65.08

per param

Table 3: This table compares the performance of E5+MMR against E5 across three different datasets, focusing on the recall of the correct answer within the selected documents.

Natural Question	GPT4			GPT3.5		
	$T_{\max}=120k$	$T_{\max}=50k$	$T_{\max}=20k$	$T_{\max}=10k$	$T_{\max}=5k$	$T_{\max}=2k$
E5 (index sort)	45.7	50.5	52.7	45.9	49.2	45.4
E5 (sort)	55.2	56.6	54.6	51.5	50.5	47.8
E5 (1:1)	53.5	55.9	55.8	50.5	50.9	46.9
E5 (2:1)	54.5	57.5	56.4	51	50.5	47.1
E5 (3:1)	54.7	57	54.8	51.3	49.8	47.4
E5+MMR (index sort)	46.2	50.3	53.2	47.6	50.9	48.6
E5+MMR (sort)	56.4	57.9	57	51.3	51.4	47.7
E5+MMR (1:1)	55	57.2	55.9	50.8	50.7	47.8
E5+MMR (2:1)	57.2	55.3	56.2	50.7	52.2	48.7
E5+MMR (3:1)	56.3	56.4	55.6	51.6	52.3	47.4

Table 4: This table compares the performance of E5+MMR against E5 on Natural Question, focusing on the recall of the correct answer within the LLM responses.

Narrative Q&A	GPT4			GPT3.5		
	$T_{max}=120k$	$T_{max}=50k$	$T_{max}=20k$	$T_{max}=10k$	$T_{max}=5k$	$T_{max}=2k$
E5 (index sort)	10.37	9.15	8.54	5.18	4.57	4.88
E5 (sort)	10.67	10.59	8.23	6.1	4.57	5.18
E5 (1:1)	9.76	9.45	7.93	4.88	4.88	5.18
E5 (2:1)	10.06	9.15	7.93	5.18	3.96	5.18
E5 (3:1)	10.98	9.45	7.93	5.79	4.57	4.88
E5+MMR (index sort)	10.67	10.37	11.28	4.88	6.1	4.88
E5+MMR (sort)	12.8	10.67	10.37	6.1	6.1	4.27
E5+MMR (1:1)	11.89	10.06	11.28	6.4	6.71	4.88
E5+MMR (2:1)	11.59	10.67	10.98	6.4	5.79	5.49
E5+MMR (3:1)	11.89	10.98	10.06	6.7	7.01	4.88

Table 5: This table compares the performance of E5+MMR against E5 on Narrative Q&A, focusing on the recall of the correct answer within the LLM responses.

Trivial Q&A	GPT4			GPT3.5		
	$T_{max}=120k$	$T_{max}=50k$	$T_{max}=20k$	$T_{max}=10k$	$T_{max}=5k$	$T_{max}=2k$
E5 (index sort)	74.21	73.21	73.12	65.18	64.19	64.68
E5 (sort)	73.81	73.91	72.42	63.59	65.08	65.57
E5 (1:1)	74.4	73.12	72.92	64.29	64.29	64.98
E5 (2:1)	73.81	73.81	72.72	64.09	64.58	64.29
E5 (3:1)	73.31	74.11	72.72	63.99	64.38	64.78
E5+MMR (index sort)	74.7	74.9	73.51	66.47	66.87	64.88
E5+MMR (sort)	74.9	74.8	73.12	64.29	65.57	66.07
E5+MMR (1:1)	75.2	75.5	73.31	65.38	65.38	65.08
E5+MMR (2:1)	74.6	74.11	72.62	64.88	65.77	65.38
E5+MMR (3:1)	74.7	74.31	73.02	65.67	65.48	64.98

Table 6: This table compares the performance of E5+MMR against E5 on Trivial Q&A, focusing on the recall of the correct answer within the LLM responses.

Based on the results across various datasets, we can assert that diversity significantly enhances the recall of the correct answer within the selected document, as demonstrated in Table 1, showing an improvement of 2% to 5%. When the extracted sentences are summarized by GPT4 using the prompt shown in Figure 4, the advantages of SB+MMR and SB+FPS over SB alone remain evident, as shown in Table 2. Additionally, we observe that the performance of Trivial Q&A after LLM is better than the retrieved sentences, with a consistent result of approximately 76%. This suggests that the performance is largely influenced by the LLM, possibly due to pretraining on Trivial Q&A, even when the retrieved documents are provided. FPS, using distance as the evaluation metric, performs slightly worse than MMR, which uses cosine similarity. Moreover, MMR is faster than FPS because computing cosine similarity is quicker than Euclidean distance in Python, especially as the compression ratio increases, as shown in subfigure (b) of Figure 3. This conclusion generally holds true across different datasets. The speed advantage of MMR becomes more critical as the number of candidates increases with the dataset size. Consequently, MMR will be used in the multiple document comparison in the next section.

Inspired by the paper "Lost in the Middle" [11], we sorted the selected sentences by different methods. The term "index sort" refers to sorting the sentences in their original order within the document. In comparison, "SB (m:n)" refers to allocating the first selected m sentences with highest scores at the beginning, the next n sentences with highest scores at the end, and then another m sentences at the beginning, continuing this pattern until all sentences are allocated. Specifically, "SB (sort)" is equivalent to "SB (1:0)" and does not alter the sequence of selected sentences. As shown in Table 2, SB (index sort) performs best because the original sequential information of the selected sentences in the document, despite missing some internal information, makes the most sense for GPT-4 in downstream tasks.

3.2 Multiple Documents Q&A

For multiple documents Q&A, the same three datasets are utilized. In these datasets, the number of documents and the length of documents are relatively long, making it impractical to split each document into sentences. Instead, we follow the general framework of RAG to split each document into chunks of 512 tokens, with an overlapping ratio of 0.5 (i.e., 256 tokens) between any two adjacent chunks. To extract embeddings from these chunks and adhere to the standard pipeline of RAG, we apply the E5 model [19]. After applying the chunking strategy, the number of chunks can still reach nearly 1 million, which is impractical for exact search. To facilitate approximate search, principal component

su tutti i docu? Chunk per docu?

analysis (PCA) [20] is first applied to reduce the dimensionality of the embeddings, followed by clustering [21] to ensure the average number of chunks is less than 10k. Unlike single document Q&A, we set the maximum number of tokens rather than the compression ratio as the threshold for the maximum number of tokens selected. Specifically, T_{max} is set to 2k, 5k, or 10k for GPT-3.5 and 20k, 50k, or 120k for GPT-4. Other settings remain the same. Different hyperparameters for α and w are tested. For the Natural Questions dataset, $\alpha = 0.9$ and $w = 5$ yield the best results for GPT-3.5, while $\alpha = 0.7$ and $w = 5$ are optimal for GPT-4 in MMR. For Narrative Q&A, $\alpha = 0.8$ and $w = 30$ are best for GPT-3.5, and $\alpha = 0.7$ and $w = 300$ are best for GPT-4 in MMR. For Trivia Q&A, $\alpha = 0.7$ and $w = 20$ are best for GPT-3.5, and $\alpha = 0.8$ and $w = 300$ are best for GPT-4 in MMR. From the results, we observe that the optimal values for α and w are generally larger for multiple document Q&A compared to single document Q&A.

→ When evaluating the performance of multiple-document Q&A systems, we observe a pattern similar to that of single-document Q&A. Specifically, the E5+MMR method shows a significant improvement over E5 in recall of the answers in retrieved documents, as demonstrated in Table 3, with a margin exceeding 10%. Additionally, E5+MMR outperforms E5 for post-LLM recall as shown in Tables 4, 5, and 6. However, future research should prioritize enhancing the LLM's ability to utilize the retrieved documents effectively, rather than merely focusing on retrieving more accurate documents, as the LLM itself is the bottleneck. This observation is further corroborated in Trivial Q&A, where the results consistently achieve 64% accuracy for GPT3.5 and 76% for GPT4, irrespective of the retrieved document. Last, unlike single-document Q&A, placing important chunks at the beginning and ending positions of the prompt can provide benefits, particularly in Natural Question scenarios, as shown in Table 4, which can lead to a 10% improvement. This finding aligns with the conclusions of the paper "Lost in the Middle". The most relevant chunks to the query should be positioned either at the beginning or the end of the prompt.

SquAD	Sentence			Chunk size: 256			Chunk size: 512		
	10k	5k	2k	10k	5k	2k	10k	5k	2k
SB/E5	86.8	83.7	78.5	95	92.7	86.3	96.7	94.3	86.6
SB/E5+MMR	90.1	89.4	86.6	97	96.6	95.4	99	97.8	96.7

Table 7: This table compares the performances of sentence splitter and chunk splitter of size 256 and 512 on SquAD, focusing on the recall of the correct answer within the selected documents.

SquAD	Sentence			Chunk size: 256			Chunk size: 512		
	10k	5k	2k	10k	5k	2k	10k	5k	2k
SB/E5 (index sort)	70.4	73	71.7	78.2	81.1	79.2	80.2	82.9	78.2
SB/E5 (sort)	71.3	70.1	67.9	79.9	80	76.6	82.9	82.2	76.5
SB/E5 (1:1)	72	71.8	69.5	80.3	80.4	77.5	82.7	82	77.4
SB/E5 (2:1)	71.9	71.7	69.4	80.4	80.1	77.4	82.9	81.9	77.5
SB/E5 (3:1)	71.8	71	68.7	81.4	79.9	77.7	82.7	82.2	77.3
SB/E5+MMR (index sort)	68.7	74.2	75	76.5	81.4	83	76.1	84.2	84.9
SB/E5+MMR (sort)	71.5	71.8	72.5	82	81.9	82.6	84.1	84.5	84.7
SB/E5+MMR (1:1)	72.5	73.6	72.9	81.3	82.2	81.7	82.3	82.9	84.6
SB/E5+MMR (2:1)	71.9	73.1	73.2	81.8	82.5	83.1	84.5	84.5	85
SB/E5+MMR (3:1)	71.7	73.4	71.9	81.4	82.5	82.9	83.4	84.3	84.5

Table 8: This table compares the performances of sentence splitter and chunk splitter of size 256 and 512 on SquAD, focusing on the recall of the correct answer within the LLM responses.

Summarization Datasets	gov_report		legal	
	Rouge	GPT4 WR	Rouge	GPT4 WR
SB	17.7	24.24	11.3	35.82
SB+MMR	18	$\frac{72.65+77.86}{2} = \mathbf{75.26}$	11.8	$\frac{71.64+56.72}{2} = \mathbf{64.18}$

Table 9: This table compares the performance of SB+MMR against SB on gov_report and legal, using ROUGE and LLM-as-a-judge.

3.3 Sentence and Chunk Splitter Comparison on SquAD

For the comparison between sentence and chunk splitters on multiple documents Q&A, only the SQuAD dataset will be considered. The dataset sizes for Natural Questions, TriviaQA, and NarrativeQA are too large, making sentence-level experiments difficult. For the sentence-level splitter, Spacy is used. For the chunk-level splitter, a threshold of 256 or 512 tokens with 50% overlap between adjacent chunks is applied. All segmented sentences or chunks are mixed,

reduced in dimension through PCA, and clustered for downstream tasks. Similar to previous experiments, T_{max} is set to 2k, 5k, or 10k for GPT3.5. For the sentence-level splitter, the best parameters are $\alpha = 0.25$ and $w = 1000$. For the 256-token chunk-level splitter, the best parameters are $\alpha = 0.5$ and $w = 300$. For the 512-token chunk-level splitter, the best parameters are $\alpha = 0.3$ and $w = 300$. The results are consistent with previous findings. SB/E5+MMR significantly outperforms SB/E5, as shown in Table 7, with a 10% increase in recall of the correct answer within the selected documents. This recall increment of SB/E5+MMR over SB/E5 still exists in the LLM response, as shown in Table 8. "Index sort" generally performs better for sentence-level splitting, while sorting based on score is usually beneficial for chunking. A new takeaway is that chunk-level performance is better than sentence-level, with even better results for larger chunk sizes.

3.4 Single Document Summarization

For single document summarization, we include two datasets: the gov report [22] and legal documents [23]. We utilize GPT3.5 for summarization. To achieve this, we filter examples that are less than 15k tokens and then apply MMR to select sentences within each document until it reaches the predetermined threshold of 8k tokens. For the 1. gov report, the best parameters are $\alpha = 0.9$ and $w = 10$. For the legal documents, the best parameters are $\alpha = 0.925$ and $w = 300$. After selecting and ordering the selected sentences based on their original sequence, they are sent to GPT3.5 to generate the final summary using a specific prompt in Figure. 5. For both datasets, expert-written golden summaries are provided for each document. We evaluate the quality of generated summary using the ROUGE score by comparing with the golden summary. In addition, summaries by SB and SB+MMR are compared using LLM-as-a-Judge through GPT4. To address the position bias problem, we switch the sequences of the two summaries in two runs and average the win rate (WR). Our experiments reveal that diversity improves summary quality, as indicated by increased ROUGE scores and a higher LLM-as-a-Judge WR. Additionally, experiments on our internal data show that diversity is particularly beneficial for long emails, articles, and logs, where redundancy is a significant issue due to repetitive content, greetings, and long URLs. Diversity avoid overestimating information similar to the query.

4 Related Works

4.1 Surpass the Context Window Limitation

The quadratic time complexity $O(N^2)$ of self-attention in transformer [1] poses limitations on training larger context window [24, 3]. A wide range of approaches has been explored to extend the context window of these models. One approach, exemplified by Longformer and its subsequent iterations [7], aims to handle much longer contexts by modifying the self-attention mechanism. Another strategy involves training models on shorter sequences and then modifying the Transformer’s structure to accommodate longer sequences during inference [25]. Lastly, some researchers are exploring methods such as chunking, leveraging discourse structures, and extracting key content to effectively reduce the context window required by LLMs, thereby facilitating the summarization of extensive documents [26]. Our paper will focus on the extraction method.

4.2 RAG & Long Document Summary

To give LLM the capability of utilizing unknown data, RAG is developed to retrieve relevant materials and utilize them for downstream tasks [4]. Several downstream approaches enhance RAG from various perspectives including graphRAG [27], correctiveRAG [28], hierarchical RAG [29], modular RAG [30] and more [31].

Long document summary includes extractive based method and abstractive based method. Initially, extractive methods, such as unsupervised Tf-Idf [32] and supervised BERT [2], were widely adopted until the advent of more potent decoder-based models. Subsequently, abstractive approaches gained prominence, exemplified by the end-to-end method [33] and hybrid extractive-abstractive strategies[34, 35, 36, 37].

Recently, Hypothetical Document Embedding (HyDE) applied LLM reranking to enhance diversity in Q&A tasks, claiming their method outperforms MMR, which provides nearly no benefit on downstream tasks [38, 39]. However, utilizing LLM for reranking can be expensive, slow and costly. In addition, these studies did not discuss the recall of relevant documents prior to LLM generation and explore various hyperparameters within MMR. This motivates us to explore more on MMR and FPS to enhance diversity, thereby improving the performance of LLMs on downstream tasks.

4.3 Representative Sentences Extraction

In the realm of extraction-based summarization, the criterion for choosing key sentences is crucial. Tf-Idf selected sentences based on the frequency of keyword [32]. PEGASUS selected sentences that utilizes structural information and ROUGE scores [34]. BERT selected representative sentences by distance in embedding space [2]. SentenceBERT [18], DeBERTa [40], E5 [19] and DPR [41] have been trained on larger corpora using more powerful computational resources. What is more, these methods only focus on selecting sentences that are related with core idea of the document, and it will cause problem for lacking the diversity of selected sentences. This research primarily focuses on leveraging smaller encoder-only models while incorporating diversity as a key consideration.

5 Conclusion

This study proves the benefits of diversity through MMR and FPS to LLM performances on Q&A and summarization. From the retrieval viewpoint, the recall is greatly improved both for sentence and chunk-level splitter, especially when α and w are properly selected. This recall rate increment is maintained after LLM generation. However, future research should pay more attention to improve the LLM’s capability to find answers from the retrieved documents. MMR shows slightly better performances compared with FPS, and its latency property is much better, which greatly increase the potential of usage in application. For sentence-level splitter, arranging the selected sentences in their original sequence is usually beneficial and for chunk-level splitter, putting more important chunks at the beginning and ending positions are beneficial. Lastly, given a multiple document Q&A like SquAD, chunk-level splitter usually has a better performance compared with sentence-level splitter. Lastly, these conclusion on Q&A can be extended to summarization task.

6 Limitation

There are several limitation on this works. To begin with, we only work on English dataset, while multilingual datasets should be tested to prove the importance of diversity on other language. In addition, this work focuses on research dataset while more work is supposed to be conducted on industrial datasets. Next, for extremely large dataset, more engineering work on parallelization like tree structures should be conducted to reduce latency. Finally, further research is needed to enhance LLMs’ ability to effectively utilize retrieved documents—such as through post-training methods that encourage the model to place greater emphasis on the retrieved information.

References

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.
- [3] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Łukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Łukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kosic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning,

- Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O’Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorny, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. Gpt-4 technical report, 2024.
- [4] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2021.
 - [5] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2024.
 - [6] Nvidia, :, Bo Adler, Niket Agarwal, Ashwath Aithal, Dong H. Anh, Pallab Bhattacharya, Annika Brundyn, Jared Casper, Bryan Catanzaro, Sharon Clay, Jonathan Cohen, Sirshak Das, Ayush Dattagupta, Olivier Delalleau, Leon Derczynski, Yi Dong, Daniel Egert, Ellie Evans, Aleksander Ficek, Denys Fridman, Shaona Ghosh, Boris Ginsburg, Igor Gitman, Tomasz Grzegorzec, Robert Hero, Jining Huang, Vibhu Jawa, Joseph Jennings, Aastha Jhunjhunwala, John Kamalu, Sadaf Khan, Oleksii Kuchaiev, Patrick LeGresley, Hui Li, Jiwei Liu, Zihan Liu, Eileen Long, Ameya Sunil Mahabaleshwarkar, Somshubra Majumdar, James Maki, Miguel Martinez, Maer Rodrigues de Melo, Ivan Moshkov, Deepak Narayanan, Sean Narenthiran, Jesus Navarro, Phong Nguyen, Osvald Nitski, Vahid Noroozi, Guruprasad Nutheti, Christopher Parisien, Jupinder Parmar, Mostofa Patwary, Krzysztof Pawelec, Wei Ping, Shrimai Prabhumoye, Rajarshi Roy, Trisha Saar, Vasanth Rao Naik Sabavat, Sanjeev Satheesh, Jane Polak Scowcroft, Jason Sewall, Pavel Shamis, Gerald Shen, Mohammad Shoeby, Dave Sizer, Misha Smelyanskiy, Felipe Soares, Makesh Narsimhan Sreedhar, Dan Su, Sandeep Subramanian, Shengyang Sun, Shubham Toshniwal, Hao Wang, Zhilin Wang, Jiaxuan You, Jiaqi Zeng, Jimmy Zhang, Jing Zhang, Vivienne Zhang, Yian Zhang, and Chen Zhu. Nemotron-4 340b technical report, 2024.
 - [7] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer, 2020.
 - [8] Hongye Jin, Xiaotian Han, Jingfeng Yang, Zhimeng Jiang, Zirui Liu, Chia-Yuan Chang, Huiyuan Chen, and Xia Hu. Llm maybe longlm: Self-extend llm context window without tuning, 2024.
 - [9] Jaime Carbonell and Jade Goldstein. The use of mmr, diversity-based reranking for reordering documents and producing summaries. In *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR ’98, page 335–336, New York, NY, USA, 1998. Association for Computing Machinery.
 - [10] Charles R. Qi, Li Yi, Hao Su, and Leonidas J. Guibas. Pointnet++: Deep hierarchical feature learning on point sets in a metric space, 2017.
 - [11] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts, 2023.
 - [12] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, Barcelona, Spain, July 2004. Association for Computational Linguistics.
 - [13] Aliyah R. Hsu, James Zhu, Zhichao Wang, Bin Bi, Shubham Mehrotra, Shiva K. Pentala, Katherine Tan, Xiang-Bo Mao, Roshanak Omrani, Sougata Chaudhuri, Regunathan Radhakrishnan, Sitaram Asur, Claire Na Cheng, and Bin Yu. Rate, explain and cite (rec): Enhanced explanation and attribution in automatic evaluation by large language models, 2024.
 - [14] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei

- Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019.
- [15] Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension, 2017.
- [16] Tomáš Kočiský, Jonathan Schwarz, Phil Blunsom, Chris Dyer, Karl Moritz Hermann, Gábor Melis, and Edward Grefenstette. The narrativeqa reading comprehension challenge, 2017.
- [17] Matthew Honnibal, Ines Montani, Sofie Van Landeghem, and Adriane Boyd. spaCy: Industrial-strength Natural Language Processing in Python. 2020.
- [18] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks, 2019.
- [19] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training, 2024.
- [20] Andrzej Maćkiewicz and Waldemar Ratajczak. Principal components analysis (pca). *Computers & Geosciences*, 19(3):303–342, 1993.
- [21] Christopher M Bishop and Nasser M Nasrabadi. *Pattern recognition and machine learning*, volume 4. Springer, 2006.
- [22] Luyang Huang, Shuyang Cao, Nikolaus Parulian, Heng Ji, and Lu Wang. Efficient attentions for long document summarization, 2021.
- [23] Abhay Shukla, Paheli Bhattacharya, Soham Poddar, Rajdeep Mukherjee, Kripabandhu Ghosh, Pawan Goyal, and Saptarshi Ghosh. Legal case document summarization: Extractive and abstractive methods and their evaluation, 2022.
- [24] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022.
- [25] Ofir Press, Noah A. Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation, 2022.
- [26] Huan Yee Koh, Jiaxin Ju, Ming Liu, and Shirui Pan. An empirical survey on long document summarization: Datasets, models, and metrics. *ACM Computing Surveys*, 55(8):1–35, December 2022.
- [27] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitan, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization, 2025.
- [28] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective retrieval augmented generation, 2024.
- [29] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. Raptor: Recursive abstractive processing for tree-organized retrieval, 2024.
- [30] Yunfan Gao, Yun Xiong, Meng Wang, and Haofen Wang. Modular rag: Transforming rag systems into lego-like reconfigurable frameworks, 2024.
- [31] Huaxia Li, Haoyun (Harry) Gao, Chengzhang Wu, and Miklos A. Vasarhelyi. Extracting financial data from unstructured sources: Leveraging large language models. *Journal of Information Systems*, 39(1):135–156, 02 2025.
- [32] Xinnian Liang, Shuangzhi Wu, Mu Li, and Zhoujun Li. Improving unsupervised extractive summarization with facet-aware modeling. In Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli, editors, *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 1685–1697, Online, August 2021. Association for Computational Linguistics.
- [33] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension, 2019.
- [34] Jingqing Zhang, Yao Zhao, Mohammad Saleh, and Peter J. Liu. Pegasus: Pre-training with extracted gap-sentences for abstractive summarization, 2020.
- [35] Ruifeng Yuan, Zili Wang, Ziqiang Cao, and Wenjie Li. Preserve context information for extract-generate long-input summarization framework. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(11):13932–13939, Jun. 2023.

- [36] Qianqian Xie, Jimin Huang, Tulika Saha, and Sophia Ananiadou. Gretel: Graph contrastive topic enhanced language model for long document extractive summarization, 2022.
- [37] Jinhyeong Lim and Hyun-Je Song. Improving multi-stage long document summarization with enhanced coarse summarizer. In Yue Dong, Wen Xiao, Lu Wang, Fei Liu, and Giuseppe Carenini, editors, *Proceedings of the 4th New Frontiers in Summarization Workshop*, pages 135–144, Singapore, December 2023. Association for Computational Linguistics.
- [38] Matouš Eibich, Shivay Nagpal, and Alexander Fred-Ojala. Aragog: Advanced rag output grading, 2024.
- [39] Marc Pickett, Jeremy Hartman, Ayan Kumar Bhowmick, Raquib ul Alam, and Aditya Vempaty. Better rag using relevant information gain, 2024.
- [40] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. Deberta: Decoding-enhanced bert with disentangled attention, 2021.
- [41] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen tau Yih. Dense passage retrieval for open-domain question answering, 2020.

A Prompts for Q&A and Summarization

Here are the prompts for Q&A in Figure. 4 and summarization in Figure. 5.

You are tasked with answering a query based on the provided context.
Respond concisely by directly citing a relevant portion of the original context.

```
query:
###
{query}
###

context:
###
{context}
###
```

answer (exactly copy from the context):

Figure 4: Prompts for Q&A

Please summarize based on the following context.
The summary should incorporate both qualitative and quantitative information.
The qualitative section should highlight central themes, emerging trends, and critical elements.
Meanwhile, the quantitative section should present supporting statistics and numerical data relevant to the summary.

```
Context:
###
{context}
###

Summary:
```

Figure 5: Prompts for Summarization

B Impacts of Hyperparameters in Diversity for Retrieval

The impact of different hyperparameters: α , w , c_r on the recall of the Natural Question dataset of single document Q&A is shown in Figure. 6. The first and second subfigures illustrate the recall ratios of answers contained in the selected documents for SB+MMR and SB+FPS. When the weight parameter $w = 1$, they are equivalent to SB. From the results, we can conclude that both SB+MMR and SB+FPS outperform SB. The last two subfigures display the latency of SB+MMR and SB+FPS. SB+FPS shows slightly worse performances than SB+MMR, and the latency of SB+MMR is significantly lower, especially when the context window is very long. Considering these two aspects, SB+MMR is more suitable for practical use compared to SB+FPS.

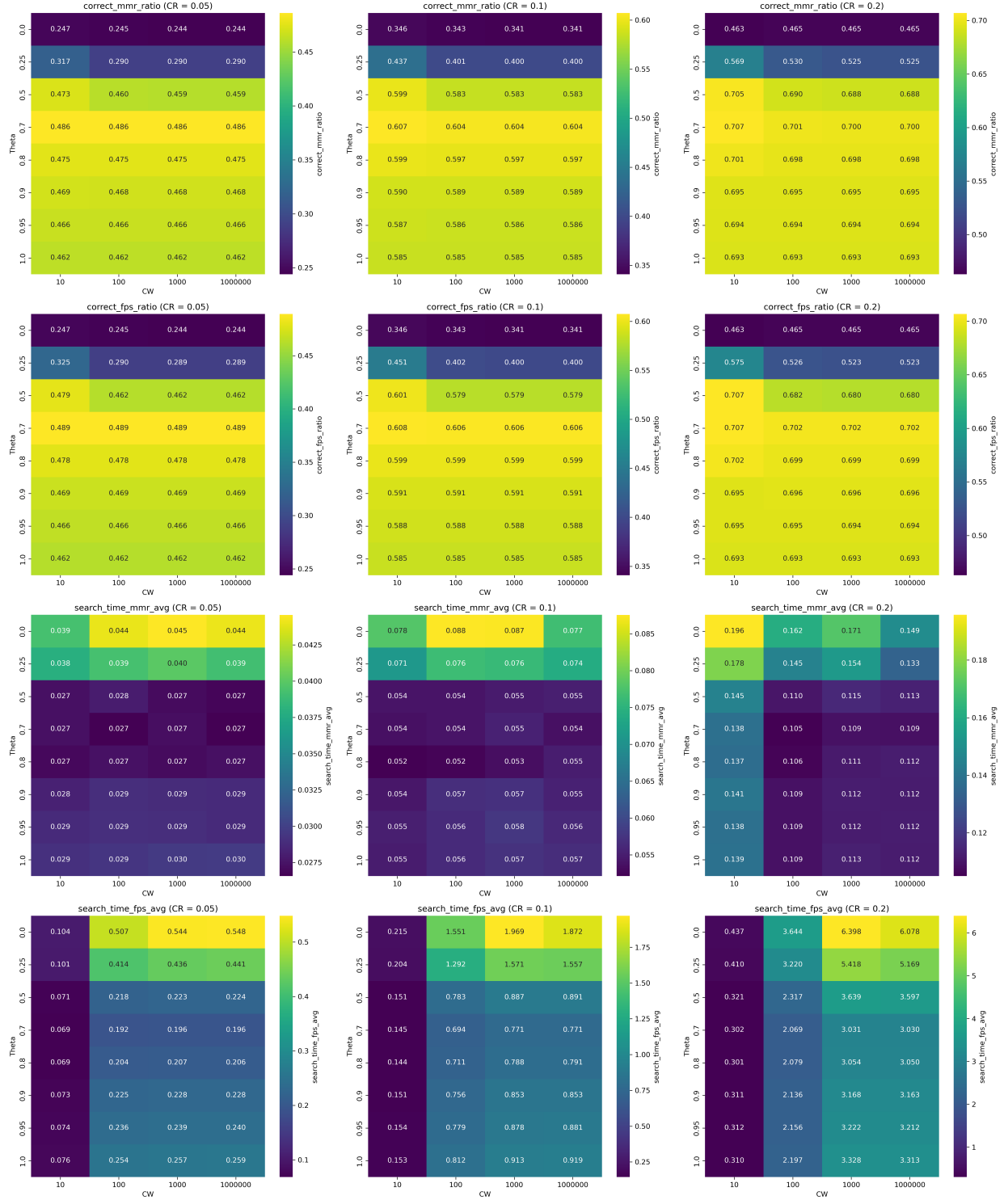


Figure 6: The impact of different hyperparameters: α , w , c_r on the recall of the Natural Question dataset of single document Q&A.